

[Open in app](#)

b

You're reading for free via [Alpha Iterations'](#) Friend Link. [Upgrade](#) to access the best of Medium.

★ Member-only story

Agentic AI Project: Build a Multi-Agent Text2SQL Chatbot for E-commerce Database

Building an Intelligent Text-to-SQL Multi-Agent Chatbot with LangGraph and Chainlit, leveraging the power of Agentic AI for E-Commerce Use Case [Code Included]

26 min read · Nov 19, 2025



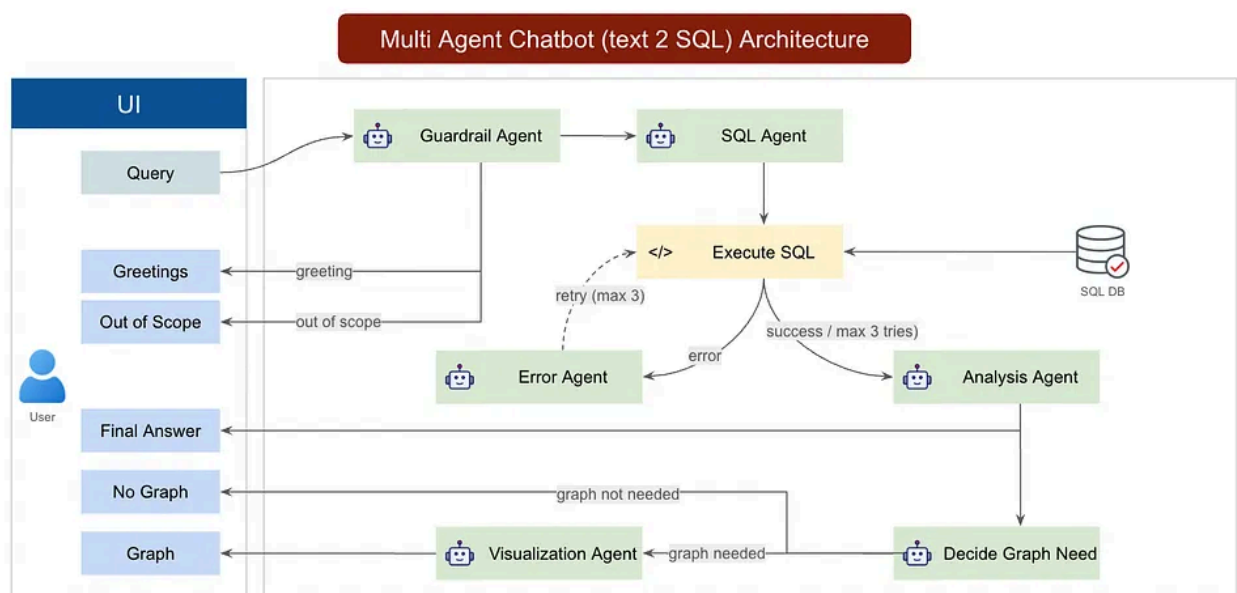
Alpha Iterations

[Follow](#)

Listen



Share

[More](#)

Non-members [read here for free.](#)

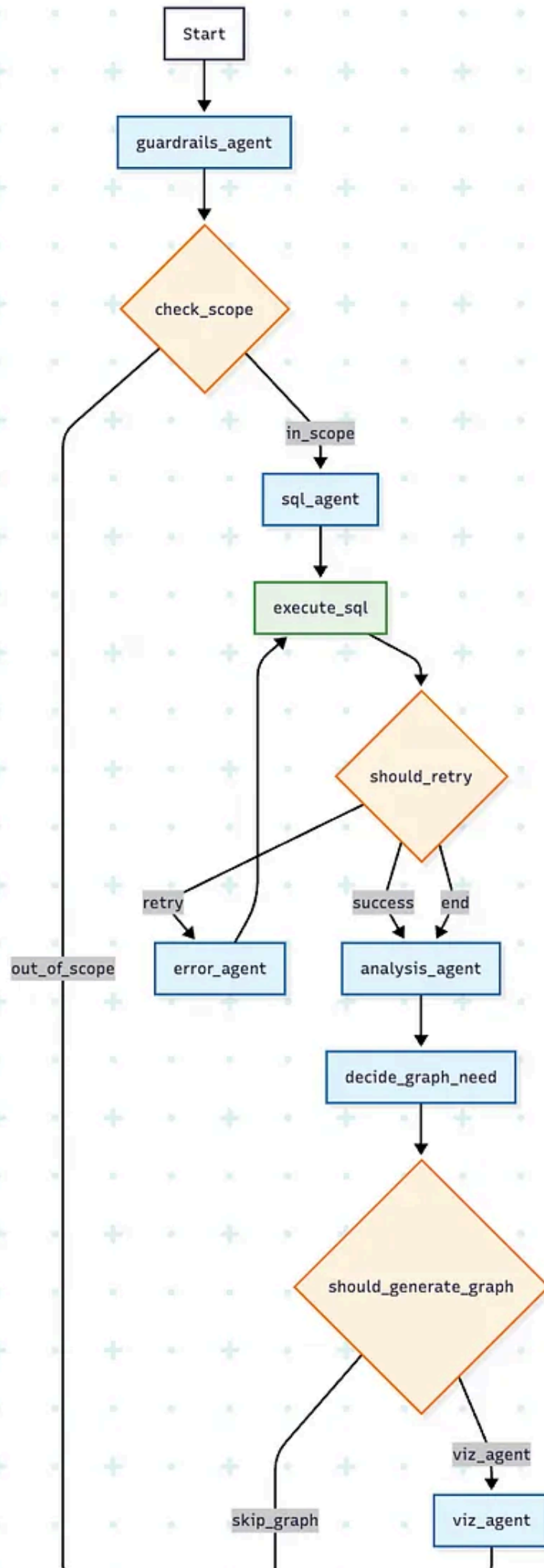
Introduction

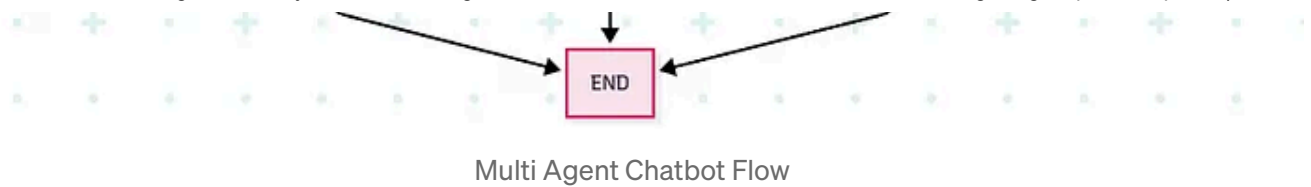
Not very long ago, only handful of people (Data Engineers, Analysts) could query the SQL databases in an organization. The people who needed it the most: Sales and Marketing team, didn't have the skills to query the databases. They had to rely on the technical teams to get some information.

This multi-agent text-2-sql chatbot removes that dependency. It empowers all the people in the organization to ask questions in natural english language.

This project demonstrate that we can democratize data access by converting natural language questions into SQL queries, executing them, and presenting results with intelligent visualizations.

Built using LangGraph's state machine architecture and powered by OpenAI's gpt-4o-mini, this system orchestrates multiple specialized AI agents that work together to provide accurate, context-aware responses to e-commerce database queries.





Problem Statement

The Challenge

Organizations maintain vast amounts of data in relational databases, but accessing this data presents several challenges:

1. **Technical Barrier:** Non-technical stakeholders cannot query databases directly
2. **SQL Complexity:** Writing complex JOINS and aggregations requires expertise
3. **Time Consumption:** Back-and-forth with data teams slows decision-making
4. **Visualization Gap:** Raw query results don't provide immediate insights
5. **Error Prone:** Manual SQL queries are susceptible to syntax and logical errors

The Solution

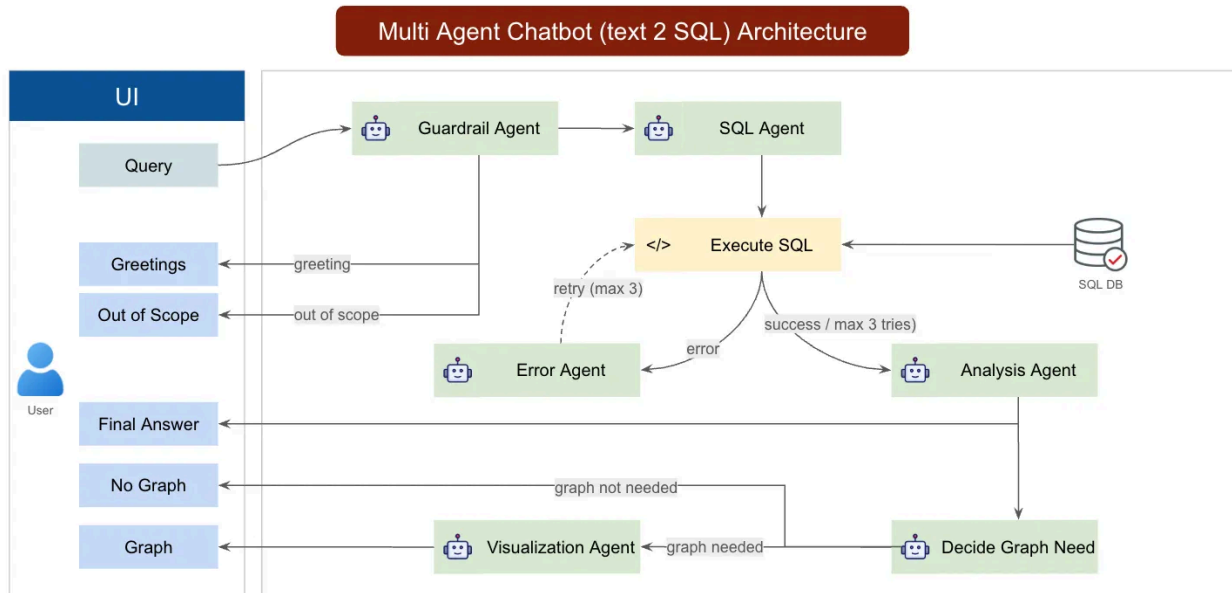
A conversational AI system that:

- Accepts natural language questions in plain English
- Validates scope to prevent irrelevant queries
- Generates optimized SQL queries automatically
- Handles errors intelligently with retry mechanisms
- Provides natural language explanations of results
- Creates visualizations when beneficial
- Streams execution process for transparency

Tech Stack & Architecture

- **Core Framework** → LangGraph
- **Front End:** Chainlit
- **AI & LLM** → OpenAI API (gpt-4-mini)

- **Data & Database** → SQLite3, Pandas, Brazilian E-Commerce Dataset (Olist)
- **Visualization** → Plotly, LLM-Generated Visualization Code
- **Utilities** → python-dotenv



Multi Agent Chatbot Architecture [Image by Author]

About Data

We are going to use Brazilian E-Commerce Public Dataset by Olist which is download from Kaggle.

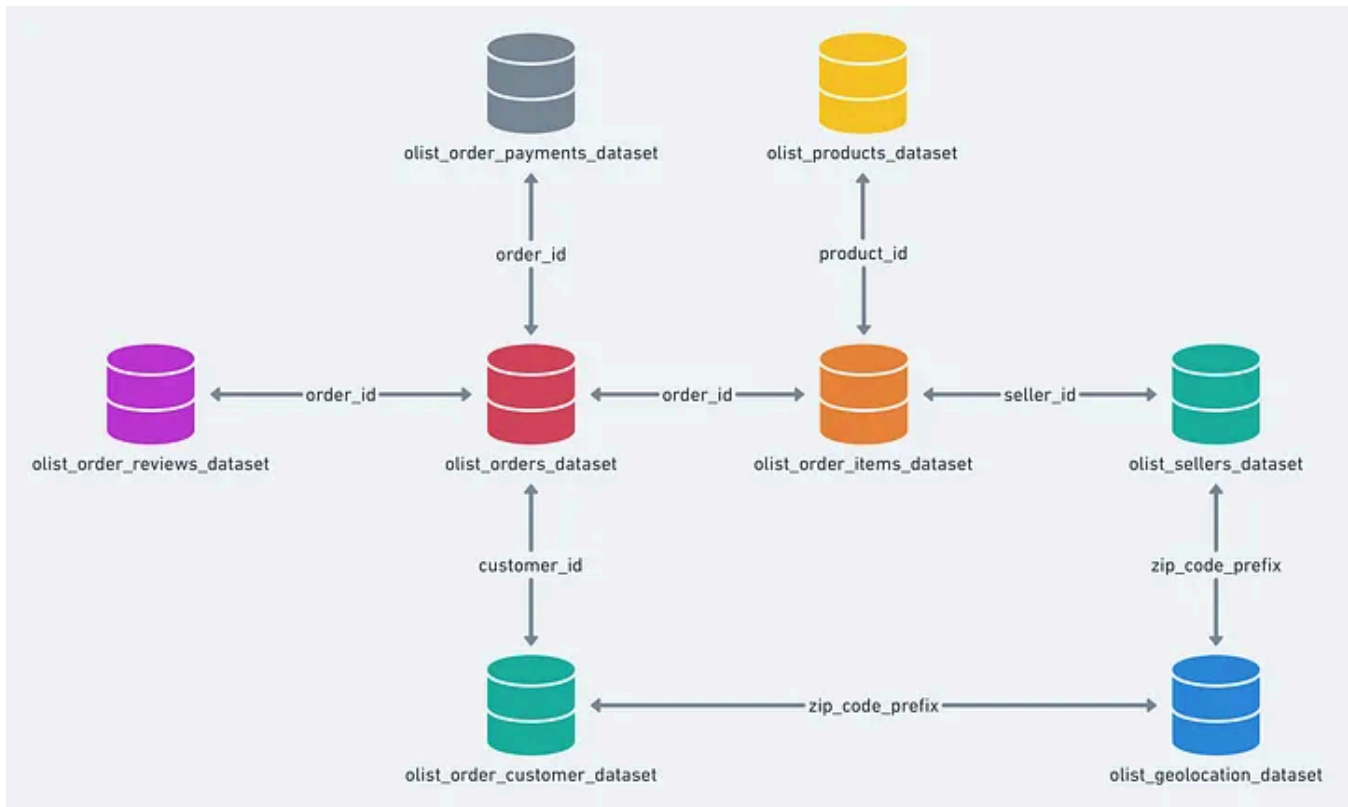
This dataset has details of 100k orders which took place between 2016 to 2018 at olist store in Brazil.

This dataset has 9 tables, which covers order details, customer details, payment details etc.

The data is anonymized:

1. An order might have multiple items.
2. Each item might be fulfilled by a distinct seller.
3. All text identifying stores and partners where replaced by the names of Game of Thrones great houses.

Below is the schema of the data:



E-Commerce Data Schema [Source: [Kaggle](#)]

Download the Data:

We download the data from [Kaggle](#) using `kagglehub` library and save it to `data` folder in the repo.

```

import kagglehub
import shutil
import os

# Download latest version
path = kagglehub.dataset_download("olistbr/brazilian-ecommerce")

# Create data folder if it doesn't exist
data_folder = "data"
os.makedirs(data_folder, exist_ok=True)

# Copy all files from the downloaded path to the data folder
for item in os.listdir(path):
    source = os.path.join(path, item)
    destination = os.path.join(data_folder, item)

    if os.path.isfile(source):
        shutil.copy2(source, destination)
        print(f"Copied: {item}")
  
```

```
print(f"\nAll files saved to '{data_folder}' folder")
```

```
Copied: olist_sellers_dataset.csv
Copied: product_category_name_translation.csv
Copied: olist_orders_dataset.csv
Copied: olist_order_items_dataset.csv
Copied: olist_customers_dataset.csv
Copied: olist_geolocation_dataset.csv
Copied: olist_order_payments_dataset.csv
Copied: olist_order_reviews_dataset.csv
Copied: olist_products_dataset.csv
```

All files saved to 'data' folder

Exploring the Dataset Files

We now check the each files one by one.

1. Customer Data

```
# Customers Dataset
customers_df = pd.read_csv('data/olist_customers_dataset.csv')
print(f"Shape: {customers_df.shape}")
customers_df.head()
```

Shape: (99441, 5)

	customer_id	customer_unique_id	customer_zip_code_prefix	customer_city	customer_state
0	06b8999e2fba1a1fbc88172c00ba8bc7	861eff4711a542e4b93843c6dd7febb0	14409	franca	SP
1	18955e83d337fd6b2def6b18a428ac77	290c77bc529b7ac935b93aa66c333dc3	9790	sao bernardo do campo	SP
2	4e7b3e00288586ebd08712fdd0374a03	060e732b5b29e8181a18229c7b0b2b5e	1151	sao paulo	SP
3	b2b6027bc5c5109e529d4dc6358b12c3	259dac757896d24d7702b9acbbff3f3c	8775	mogi das cruzeiras	SP
4	4f2d8ab171c80ec8364f7c12e35b23ad	345ecd01c38d18a9036ed96c73b8d066	13056	campinas	SP

First 5 rows of the Customer Data

2. Geo Location Data

Geolocation Dataset

```
geolocation_df = pd.read_csv('data/olist_geolocation_dataset.csv')
print(f"Shape: {geolocation_df.shape}")
geolocation_df.head()
```

Shape: (1000163, 5)

	geolocation_zip_code_prefix	geolocation_lat	geolocation_lng	geolocation_city	geolocation_state
0	1037	-23.545621	-46.639292	sao paulo	SP
1	1046	-23.546081	-46.644820	sao paulo	SP
2	1046	-23.546129	-46.642951	sao paulo	SP
3	1041	-23.544392	-46.639499	sao paulo	SP
4	1035	-23.541578	-46.641607	sao paulo	SP

First 5 rows of the Geo Location Data

3. Order Items Dataset

Order Items Dataset

```
order_items_df = pd.read_csv('data/olist_order_items_dataset.csv')
print(f"Shape: {order_items_df.shape}")
order_items_df.head()
```

Shape: (112650, 7)

	order_id	order_item_id	product_id	seller_id	shipping_limit_date	price	freight_value
0	00010242fe8c5a6d1ba2dd792cb16214	1	4244733e06e7ecb4970a6e2683c13e61	48436dade18ac8b2bee089ec2a041202	2017-09-19 09:45:35	58.90	13.29
1	00018f77f2f0320c557190d7a144bdd3	1	e5f2d52b802189ee658865ca93d83a8f	dd7ddc04e1b6c2c614352b383efe2d36	2017-05-03 11:05:13	239.90	19.93
2	000229ec398224ef6ca0657da4fc703e	1	c777355d18b72b67abbeef9df44fd0fd	5b51032eddd242adc84c38acab88f23d	2018-01-18 14:48:30	199.00	17.87
3	00024acbcdff0a6daa1e931b038114c75	1	7634da152a4610f1595efa32f14722fc	9d7a1d34a5052409006425275ba1c2b4	2018-08-15 10:10:18	12.99	12.79
4	00042b26cf59d7ce69dfabb4e55b4fd9	1	ac6c3623068f30de03045865e4e10089	df560393f3a51e74553ab94004ba5c87	2017-02-13 13:57:51	199.90	18.14

First 5 rows of the Order Items Data

4. Order Payments Dataset

Order Payments Dataset

```
order_payments_df = pd.read_csv('data/olist_order_payments_dataset.csv')
print(f"Shape: {order_payments_df.shape}")
order_payments_df.head()
```

Shape: (103886, 5)

	order_id	payment_sequential	payment_type	payment_installments	payment_value
0	b81ef226f3fe1789b1e8b2acac839d17	1	credit_card	8	99.33
1	a9810da82917af2d9aefd1278f1dcfa0	1	credit_card	1	24.39
2	25e8ea4e93396b6fa0d3dd708e76c1bd	1	credit_card	1	65.71
3	ba78997921bbcdc1373bb41e913ab953	1	credit_card	8	107.78
4	42fdf880ba16b47b59251dd489d4441a	1	credit_card	2	128.45

First 5 rows of the Order Payments Data

5. Order Reviews Dataset

```
# Order Reviews Dataset
order_reviews_df = pd.read_csv('data/olist_order_reviews_dataset.csv')
print(f"Shape: {order_reviews_df.shape}")
order_reviews_df.head()
```

Shape: (99224, 7)

	review_id	order_id	review_score	review_comment_title	review_comment_message	review_creation_date	review_answer_timestamp
0	7bc2406110b926393aa56f80a40eba40	73fc7af87114b39712e6da79b0a377eb	4	NaN	NaN	2018-01-18 00:00:00	2018-01-18 21:46:59
1	80e641a11e56f04c1ad469d5645dfde	a548910a1c6147796b98fdf73dbeba33	5	NaN	NaN	2018-03-10 00:00:00	2018-03-11 03:05:13
2	228ce5500dc1d8e020d8d1322874b6f0	f9e4b6558b201a9f2ecdecbb34bed034b	5	NaN	NaN	2018-02-17 00:00:00	2018-02-18 14:36:24
3	e64fb393e7b32834bb789ff8bb30750e	658677c97b385a9be170737859d3511b	5	NaN	Recebi bem antes do prazo estipulado.	2017-04-21 00:00:00	2017-04-21 22:02:06
4	f7c4243c7fe1938f181bec41a392bdeb	8e6bfb81e283fa7e4f11123a3fb894f1	5	NaN	Parabéns lojas Iannister adorei comprar pela l...	2018-03-01 00:00:00	2018-03-02 10:26:53

First 5 rows of the Order Reviews Data

6. Orders Dataset

```
# Orders Dataset
orders_df = pd.read_csv('data/olist_orders_dataset.csv')
print(f"Shape: {orders_df.shape}")
orders_df.head()
```

Shape: (99441, 8)

	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_customer_date	order_estimated_delivery_date
0	e481f51c8dc54678b7cc49136f2d6af7	9e6432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15	2017-10-04 19:55:00	2017-10-10 21:26:13	2017-10-18 00:00:00
1	53c0db21c8bc79ce0b674e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-07-26 03:24:27	2018-07-26 14:31:00	2018-08-07 15:27:45	2018-08-13 00:00:00
2	47770eb9150c20cc44946d9c07ec65d	45ce2a54c0b03ef3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-08 08:55:23	2018-08-08 13:50:00	2018-08-17 18:56:29	2018-09-04 00:00:00
3	949d5e44dbf1de918fe9c16f97b458a	f88197486ea7920adcd8ec7375364682	delivered	2017-11-18 19:28:06	2017-11-18 19:45:59	2017-11-22 13:39:59	2017-12-02 00:18:42	2017-12-15 00:00:00
4	ad21c59c0840e6c83a9ce0573f8159	8ab97904e6daa8866dbdc4fbd7aad2c	delivered	2018-02-13 21:18:39	2018-02-13 22:20:29	2018-02-14 19:46:34	2018-02-16 18:17:02	2018-02-26 00:00:00

First 5 rows of the Orders Data

7. Products Dataset

```
# Products Dataset
products_df = pd.read_csv('data/olist_products_dataset.csv')
print(f"Shape: {products_df.shape}")
products_df.head()
```

Shape: (32951, 9)

	product_id	product_category_name	product_name_lenght	product_description_lenght	product_photos_qty	product_weight_g	product_length_cm	product_height_cm	product_width_cm
0	1e9e8ef04dbc1f4541ed26657ea517e5	perfumaria	40.0	287.0	1.0	225.0	16.0	10.0	14.0
1	3aa071139cb16b67ca9e5dea641aaa2f	artes	44.0	276.0	1.0	1000.0	30.0	18.0	20.0
2	96bd76ec8810374ed1b65e291975717f	esporte_lazer	46.0	250.0	1.0	154.0	18.0	9.0	15.0
3	cef67bcfe19066a932b73e239eb23d	bebes	27.0	261.0	1.0	371.0	26.0	4.0	26.0
4	9dc1a7de274444849c219cff195d5b71	utilidades_domesticas	37.0	402.0	4.0	625.0	20.0	17.0	13.0

First 5 rows of the Products Data

8. Sellers Dataset

```
# Sellers Dataset
sellers_df = pd.read_csv('data/olist_sellers_dataset.csv')
print(f"Shape: {sellers_df.shape}")
sellers_df.head()
```

Shape: (3095, 4)

	seller_id	seller_zip_code_prefix	seller_city	seller_state
0	3442f8959a84dea7ee197c632cb2df15	13023	campinas	SP
1	d1b65fc7debc3361ea86b5f14c68d2e2	13844	mogi guacu	SP
2	ce3ad9de960102d0677a81f5d0bb7b2d	20031	rio de janeiro	RJ
3	c0f3eea2e14555b6faeea3dd58c1b1c3	4195	sao paulo	SP
4	51a04a8a6bdcb23deccc82b0b80742cf	12914	braganca paulista	SP

First 5 rows of the Sellers Data

9. Product Category Translation Dataset

```
# Product Category Translation Dataset
category_translation_df = pd.read_csv('data/product_category_name_translation.csv')
print(f"Shape: {category_translation_df.shape}")
category_translation_df.head()
```

Shape: (71, 2)

	product_category_name	product_category_name_english
0	beleza_saude	health_beauty
1	informatica_acessorios	computers_accessories
2	automotivo	auto
3	cama_mesa_banho	bed_bath_table
4	moveis_decoracao	furniture_decor

First 5 rows of the Product Category Translation Data

Create the SQLite Database

We create `db_init.py` which performs following tasks:

- Checks if `ecommerce.db` exists and removes it to start fresh
- Creates a new SQLite database connection
- Loads each CSV file from the `/data` folder into a database table using pandas
- Prints a summary of rows and columns for each table, then closes the connection

```
#db_init.py
import pandas as pd
import sqlite3
import os

# Path to local data folder
data_path = "data"

db = "ecommerce.db"
if os.path.exists(db):
    os.remove(db)

conn = sqlite3.connect(db)

def load(name):
    df = pd.read_csv(f"{data_path}/{name}")
    table_name = name.replace("olist_", "").replace("_dataset.csv", "").replace(" ", "_")
    df.to_sql(table_name, conn, index=False, if_exists="replace")
    print(f"Loaded: {name} -> {table_name} table ({df.shape[0]} rows, {df.shape[1]} columns)")

# Load all datasets
load("olist_customers_dataset.csv")
load("olist_orders_dataset.csv")
```

```
load("olist_order_items_dataset.csv")
load("olist_order_payments_dataset.csv")
load("olist_order_reviews_dataset.csv")
load("olist_products_dataset.csv")
load("olist_sellers_dataset.csv")
load("olist_geolocation_dataset.csv")
load("product_category_name_translation.csv")

conn.close()
print(f"\nDatabase created: {db}")
print("Tables: customers, orders, order_items, order_payments, order_reviews, p
```

To execute the above file, please run below command in the terminal:

```
#Terminal
python db_init.py
```

```
#Output
Loaded: olist_customers_dataset.csv -> customers table (99441 rows, 5 columns)
Loaded: olist_orders_dataset.csv -> orders table (99441 rows, 8 columns)
Loaded: olist_order_items_dataset.csv -> order_items table (112650 rows, 7 columns)
Loaded: olist_order_payments_dataset.csv -> order_payments table (103886 rows, 5 columns)
Loaded: olist_order_reviews_dataset.csv -> order_reviews table (99224 rows, 7 columns)
Loaded: olist_products_dataset.csv -> products table (32951 rows, 9 columns)
Loaded: olist_sellers_dataset.csv -> sellers table (3095 rows, 4 columns)
Loaded: olist_geolocation_dataset.csv -> geolocation table (1000163 rows, 5 columns)
Loaded: product_category_name_translation.csv -> product_category_name_translation table (2911 rows, 2 columns)

Database created: ecommerce.db
Tables: customers, orders, order_items, order_payments, order_reviews, products
```

Defining the Agentic Flow

We now define `text2sql_agent.py` which defines various agents and functions as follows.

- This module builds a full **Text-to-SQL agent using LangGraph**, converting natural-language questions into SQL, executing them, and generating natural-language answers.

- It also includes **automatic error correction, scope checking, and optional Plotly graph generation**, forming a complete end-to-end e-commerce analytics assistant.

1. Importing the Libraries:

We import the required libraries. Load the `OPENAI_API_KEY` from the `.env` file.

```
#text2sql_agent.py

import os
import sqlite3
from typing import TypedDict
from langgraph.graph import StateGraph, END
from openai import OpenAI
import json
import pandas as pd

# Initialize OpenAI client
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# Database configuration
DB_PATH = "ecommerce.db"
```

2. Defining the Schema

We define the database schema as follows:

```
#text2sql_agent.py

# Database schema information
SCHEMA_INFO = """
Database Schema for E-commerce System:

1. customers
  - customer_id (TEXT): Unique customer identifier
  - customer_unique_id (TEXT): Unique customer identifier across datasets
  - customer_zip_code_prefix (INTEGER): Customer zip code
  - customer_city (TEXT): Customer city
  - customer_state (TEXT): Customer state

2. orders
  - order_id (TEXT): Unique order identifier
  - customer_id (TEXT): Foreign key to customers
  - order_status (TEXT): Order status (delivered, shipped, etc.)
  - order_purchase_timestamp (TEXT): When the order was placed
  - order_approved_at (TEXT): When payment was approved
```

- order_delivered_carrier_date (TEXT): When order was handed to carrier
- order_delivered_customer_date (TEXT): When customer received the order
- order_estimated_delivery_date (TEXT): Estimated delivery date

3. order_items

- order_id (TEXT): Foreign key to orders
- order_item_id (INTEGER): Item sequence number within order
- product_id (TEXT): Foreign key to products
- seller_id (TEXT): Foreign key to sellers
- shipping_limit_date (TEXT): Shipping deadline
- price (REAL): Item price
- freight_value (REAL): Shipping cost

4. order_payments

- order_id (TEXT): Foreign key to orders
- payment_sequential (INTEGER): Payment sequence number
- payment_type (TEXT): Payment method (credit_card, boleto, etc.)
- payment_installments (INTEGER): Number of installments
- payment_value (REAL): Payment amount

5. order_reviews

- review_id (TEXT): Unique review identifier
- order_id (TEXT): Foreign key to orders
- review_score (INTEGER): Review score (1-5)
- review_comment_title (TEXT): Review title
- review_comment_message (TEXT): Review message
- review_creation_date (TEXT): When review was created
- review_answer_timestamp (TEXT): When review was answered

6. products

- product_id (TEXT): Unique product identifier
- product_category_name (TEXT): Product category (in Portuguese)
- product_name_lenght (REAL): Product name length
- product_description_lenght (REAL): Product description length
- product_photos_qty (REAL): Number of product photos
- product_weight_g (REAL): Product weight in grams
- product_length_cm (REAL): Product length in cm
- product_height_cm (REAL): Product height in cm
- product_width_cm (REAL): Product width in cm

7. sellers

- seller_id (TEXT): Unique seller identifier
- seller_zip_code_prefix (INTEGER): Seller zip code
- seller_city (TEXT): Seller city
- seller_state (TEXT): Seller state

8. geolocation

- geolocation_zip_code_prefix (INTEGER): Zip code prefix
- geolocation_lat (REAL): Latitude
- geolocation_lng (REAL): Longitude
- geolocation_city (TEXT): City name
- geolocation_state (TEXT): State code

```

9. product_category_name_translation
    - product_category_name (TEXT): Category name in Portuguese
    - product_category_name_english (TEXT): Category name in English
"""

```

3. Agent State

De now define the Agent State using **TypedDict** format.

Using **TypedDict** instead of a normal **Dict** in LangGraph is helpful because:

1. **You can't accidentally use the wrong data:** Python will warn you if you put the wrong kind of value in the state.
2. **Your editor knows what keys exist:** so you get autocomplete and fewer mistakes when typing.

```

#text2sql_agent.py
class AgentState(TypedDict):
    """State of the agent workflow"""
    question: str
    sql_query: str
    query_result: str
    final_answer: str
    error: str
    iteration: int
    needs_graph: bool
    graph_type: str
    graph_json: str # Plotly figure JSON for Chainlit
    is_in_scope: bool # Whether the question is about e-commerce data

```

4. Agent Config

We now define `AGENT_CONFIGS` where the role and system prompt of each agent is described.

```

# text2sql_agent.py
# Agent configurations with different roles and personalities

AGENT_CONFIGS = {
    "guardrails_agent": {
        "role": "Security and Scope Manager",
        "system_prompt": "You are a strict guardrails system that filters quest

```

```
    },
    "sql_agent": {
        "role": "SQL Expert",
        "system_prompt": "You are a senior SQL developer specializing in e-commerce database queries.",
    },
    "analysis_agent": {
        "role": "Data Analyst",
        "system_prompt": "You are a helpful data analyst that explains database results and trends.",
    },
    "viz_agent": {
        "role": "Visualization Specialist",
        "system_prompt": "You are a data visualization expert. Generate clean, readable charts and graphs.",
    },
    "error_agent": {
        "role": "Error Recovery Specialist",
        "system_prompt": "You diagnose and fix SQL errors with expert knowledge and provide helpful suggestions.",
    }
}
```

5. Guardrail Agent:

Purpose

- Determines whether the question is about the e-commerce dataset.
- Distinguishes greetings, valid analytics questions, and out-of-scope queries.

Input

- question

Output

- is_in_scope (True/False)
- final_answer (If greeting or out of scope)
- Passes state unchanged if in-scope.

Notes

- If greeting → sends friendly welcome message and ends.
- If out-of-scope → returns predefined rejection message.
- Only in-scope questions continue to SQL generation.

```
#text2sql_agent.py
def guardrails_agent(state: AgentState) -> AgentState:
    """Check if the question is within scope (e-commerce related)"""
    question = state["question"]

    prompt = f"""You are a guardrails system for an e-commerce database chatbot
```

The chatbot has access to an e-commerce database with information about:

- Customers and their locations
- Orders and order status (data from 2016-2018)
- Products and categories
- Sellers
- Payments
- Reviews
- Shipping and delivery information

Examples of GREETING messages:

- "Hi", "Hello", "Hey"
- "Good morning", "Good afternoon"
- "How are you?"
- Any casual greeting or introduction

Examples of IN-SCOPE questions:

- "How many orders were placed last month?"
- "What are the top selling products?"
- "Show me customer distribution by state"
- "What is the average order value?"
- "Which sellers have the highest ratings?"

Examples of OUT-OF-SCOPE questions:

- Personal questions (e.g., "What is my wife's name?", "Where do I live?")
- Political questions (e.g., "Who should I vote for?", "What do you think about
- General knowledge (e.g., "What is the capital of France?", "How does photosyn
- Unrelated topics (e.g., "Tell me a joke", "What's the weather like?")

User Question: {question}

Analyze the question and respond in JSON format:

```
{
  "is_in_scope": true/false,
  "is_greeting": true/false,
  "reason": "brief explanation of why it is or isn't in scope or if it's a gr
}
```

If the question is a greeting, mark is_greeting as true and is_in_scope as false.
If the question is ambiguous but could potentially relate to the e-commerce database,

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": AGENT_CONFIGS["guardrails_agent"]["sy
```

```

        {"role": "user", "content": prompt}
    ],
    temperature=0,
    response_format={"type": "json_object"}
)

result = json.loads(response.choices[0].message.content)
state["is_in_scope"] = result.get("is_in_scope", False)
is_greeting = result.get("is_greeting", False)

# If it's a greeting, provide a welcome message
if is_greeting:
    state["final_answer"] = "Hi! I am your e-commerce assistant. I can answer your questions."
    return state

# If out of scope, set the final answer immediately
if not state["is_in_scope"]:
    state["final_answer"] = "I apologize, but your question appears to be out of scope."

return state

```

6. SQL Generation Agent:

Purpose

- Converts natural language into syntactically correct SQLite SQL.
- Generates **only** SQL — no markdown, no explanation.

Input

- question
- Full database schema

Output

- sql_query
- Increments iteration count

Notes

- Adds LIMIT 10 when query could be large.
- Supports multi-query output separated by semicolons.

- Must respect schema exactly.

```
#text2sql_agent.py

def sql_agent(state: AgentState) -> AgentState:
    """Generate SQL query from natural language question"""
    question = state["question"]
    iteration = state.get("iteration", 0)

    prompt = f"""You are a SQL expert. Convert the following natural language question to a SQL query.

    {SCHEMA_INFO}

    Question: {question}

    Important Guidelines:
    1. Use only the tables and columns mentioned in the schema
    2. Use proper JOIN clauses when querying multiple tables
    3. Return ONLY the SQL query without any explanation or markdown formatting
    4. If the question contains multiple sub-questions, generate separate SQL queries
    5. Use aggregate functions (COUNT, SUM, AVG, etc.) appropriately
    6. Add LIMIT clauses for queries that might return many rows (default LIMIT 10)
    7. Use proper WHERE clauses to filter data
    8. For date comparisons, remember the dates are stored as TEXT in ISO format
    9. Each SQL statement should be on its own line for clarity when multiple queries are generated

    Generate the SQL query:"""

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": AGENT_CONFIGS["sql_agent"]["system_prompt"]},
            {"role": "user", "content": prompt}
        ],
        temperature=0
    )

    sql_query = response.choices[0].message.content.strip()
    # Remove markdown code blocks if present
    sql_query = sql_query.replace("`sql", "").replace("`", "").strip()

    state["sql_query"] = sql_query
    state["iteration"] = iteration + 1

    return state
```

7. Execute SQL:

Purpose

- Runs the SQL safely against SQLite.
- Supports multi-statement SQL.

Input

- `sql_query`

Output

- `query_result` (JSON string)
- Or sets `error` on failure

Notes

- Converts rows → list of dicts.
- Limits result sets to 100 rows.
- Handles multiple queries by labeling `query_1`, `query_2`, etc.

```
#text2sql_agent.py
def execute_sql(state: AgentState) -> AgentState:
    """Execute the generated SQL query (handles multiple queries if present)"""
    sql_query = state["sql_query"]

    try:
        conn = sqlite3.connect(DB_PATH)
        cursor = conn.cursor()

        # Split multiple SQL statements (separated by semicolons)
        # Remove empty statements and strip whitespace
        sql_statements = [stmt.strip() for stmt in sql_query.split(';') if stmt]

        all_results = []

        # Execute each statement separately
        for i, statement in enumerate(sql_statements):
            cursor.execute(statement)

            # Fetch results for this statement
            results = cursor.fetchall()
```

```

    if results:
        column_names = [description[0] for description in cursor.description]

        # Convert to list of dictionaries
        formatted_results = []
        for row in results[:100]: # Limit to 100 rows per query
            formatted_results.append(dict(zip(column_names, row)))

        # If multiple queries, label them
        if len(sql_statements) > 1:
            all_results.append({
                f"query_{i+1}": formatted_results,
                f"query_{i+1}_sql": statement
            })
        else:
            all_results = formatted_results

    conn.close()

    # Format results
    if not all_results:
        state["query_result"] = "No results found."
    else:
        state["query_result"] = json.dumps(all_results, indent=2)

    state["error"] = ""

    except Exception as e:
        state["error"] = f"SQL Execution Error: {str(e)}"
        state["query_result"] = ""

    return state

```

8. Error Agent:

Purpose

- Attempts to automatically fix bad SQL.

Input

- error
- sql_query
- question

Output

- Repaired `sql_query`
- Clears `error`
- Increments `retry` iteration

Retry Logic

- Retries up to 3 times.
- If still failing after 3 attempts → returns apology in `final_answer`.

```
#text2sql_agent.py

def error_agent(state: AgentState) -> AgentState:
    """Handle errors and attempt to fix the SQL query"""
    error = state["error"]
    sql_query = state["sql_query"]
    question = state["question"]
    iteration = state.get("iteration", 0)

    # If we've tried too many times, give up
    if iteration > 3:
        state["final_answer"] = f"I apologize, but I'm having trouble generating a corrected SQL query."
        return state

    prompt = f"""The following SQL query failed with an error. Please fix it.

{SCHEMA_INFO}

Original Question: {question}

Failed SQL Query: {sql_query}

Error: {error}

Generate a corrected SQL query that will work. Return ONLY the SQL query without any extra text or formatting.

"""

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": AGENT_CONFIGS["error_agent"]["system_prompt"]},
            {"role": "user", "content": prompt}
        ],
        temperature=0
    )

    corrected_query = response.choices[0].message.content.strip()
    corrected_query = corrected_query.replace("`sql", "").replace("`", "").replace("`", "")
```

```
state["sql_query"] = corrected_query
state["error"] = "" # Clear the error for retry
state["iteration"] = iteration + 1 # Increment iteration counter

return state
```

9. Analysis Agent

Purpose

- Converts raw SQL JSON output into a readable explanation.

Input

- question
- sql_query
- query_result

Output

- final_answer

Notes

- Supports multi-query responses.
- Produces user-friendly numbered lists, insights, summaries

```
#text2sql_agent.py
def analysis_agent(state: AgentState) -> AgentState:
    """Generate natural language answer from query results"""
    question = state["question"]
    sql_query = state["sql_query"]
    query_result = state["query_result"]

    prompt = f"""You are a helpful assistant that explains database query results.

    Original Question: {question}

    SQL Query Used: {sql_query}

    Query Results:
```

```
{query_result}
```

Please provide a clear, concise answer to the original question based on the query results.
Format the answer in a user-friendly way. If the results contain numbers, present them in a readable format.
If there are multiple queries/results (for multi-part questions), address each part separately.
Use bullet points or numbered lists for multiple answers.

Answer: ""

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": AGENT_CONFIGS["analysis_agent"]["system_prompt"]},
        {"role": "user", "content": prompt}
    ],
    temperature=0
)

final_answer = response.choices[0].message.content.strip()
state["final_answer"] = final_answer

return state
```

10. Decide Graph Need:

Purpose

- Determines whether visualization adds value.

Input

- question
- query_result

Output

- needs_graph : True/False
- graph_type : "bar", "line", "pie", "scatter"

Rules

- Time trends → line
- Category comparison → bar

- Proportion → pie
- Correlation → scatter
- Single numbers → no graph

```
#text2sql_agent.py
```

```
def decide_graph_need(state: AgentState) -> AgentState:
    """Decide if a graph visualization would be helpful for the query"""
    question = state["question"]
    query_result = state["query_result"]

    # If no results or error, no graph needed
    if not query_result or query_result == "No results found." or state.get("error"):
        state["needs_graph"] = False
        state["graph_type"] = ""
        return state

    prompt = f"""Analyze the following question and query results to determine
    if a graph visualization would be helpful for the query.

    Question: {question}

    Query Results Sample:
    {query_result[:500]}...

    Determine:
    1. Would a graph be helpful for this data? (YES/NO)
    2. If yes, what type of graph? (bar, line, pie, scatter)

    Consider:
    - Trends over time → line chart
    - Comparisons between categories → bar chart
    - Proportions/percentages → pie chart
    - Correlations → scatter plot
    - Simple counts or single values → NO graph needed

    Respond in JSON format:
    {"needs_graph": true/false, "graph_type": "bar/line/pie/scatter/none", "reason": ""}

    """

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "You are a data visualization expert."},
            {"role": "user", "content": prompt}
        ],
        temperature=0,
        response_format={"type": "json_object"}
    )
```

```

decision = json.loads(response.choices[0].message.content)
state["needs_graph"] = decision.get("needs_graph", False)
state["graph_type"] = decision.get("graph_type", "none")

return state

```

11. Visualization Agent

Purpose

- Generates `Plotly` code using the LLM and executes it.
- Converts the resulting figure into JSON for frontend rendering.

Input

- `query_result`
- `graph_type`
- Internal dataframe `df`

Output

- `graph_json`

Notes

- LLM returns code only — no markdown.
- Code executed in a restricted namespace.
- Figure must be named `fig`.

```
#text2sql_agent.py
```

```

def viz_agent(state: AgentState) -> AgentState:
    """Generate a graph visualization from query results using LLM-generated Plotly code.
    query_result = state["query_result"]
    graph_type = state["graph_type"]
    question = state["question"]

    try:

```

```

# Parse query results
results = json.loads(query_result)
if not results or len(results) == 0:
    state["graph_json"] = ""
    return state

# Convert to DataFrame for context
df = pd.DataFrame(results)
columns = df.columns.tolist()
sample_data = df.head(5).to_dict('records')

# Generate Plotly code using LLM
prompt = f"""Generate Python code using Plotly to visualize the followi

```

```

Question: {question}
Graph Type: {graph_type}
Columns: {columns}
Sample Data (first 5 rows): {json.dumps(sample_data, indent=2)}
Total Rows: {len(df)}

```

Requirements:

1. Use `plotly.graph_objects` or `plotly.express`
2. The data is already loaded as 'df' (a pandas DataFrame)
3. Create an appropriate {graph_type} chart
4. Limit data to top 20 rows if there are many rows
5. Add proper titles, labels, and formatting
6. The figure variable must be named 'fig'
7. Return ONLY the Python code, no explanations or markdown
8. Do NOT include any import statements
9. Do NOT include code to show the figure (no `fig.show()`)
10. Make the visualization visually appealing with appropriate colors and layout
11. Update the layout for better interactivity (hover info, responsive sizing)

Generate the Plotly code: """

```

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": AGENT_CONFIGS["viz_agent"]["system_prompt"]},
        {"role": "user", "content": prompt}
    ],
    temperature=0
)

plotly_code = response.choices[0].message.content.strip()
# Remove markdown code blocks if present
plotly_code = plotly_code.replace("`python", "").replace("`", "").strip()

# Prepare execution environment
exec_globals = {
    'df': df,
    'pd': pd,
    'json': json
}

```

```

}

# Import plotly dynamically
try:
    import plotly.graph_objects as go
    import plotly.express as px
    exec_globals['go'] = go
    exec_globals['px'] = px
except ImportError:
    print("Plotly not installed. Installing...")
    import subprocess
    subprocess.check_call(['pip', 'install', 'plotly'])
    import plotly.graph_objects as go
    import plotly.express as px
    exec_globals['go'] = go
    exec_globals['px'] = px

# Execute the generated code
exec(plotly_code, exec_globals)

# Get the figure object
fig = exec_globals.get('fig')

if fig is None:
    raise ValueError("Generated code did not create a 'fig' variable")

# Export figure as JSON for Chainlit's Plotly element
graph_json = fig.to_json()
state["graph_json"] = graph_json

except Exception as e:
    print(f"Graph generation error: {e}")
    print(f"Generated code:\n{plotly_code if 'plotly_code' in locals() else ''}")
    state["graph_json"] = ""

return state

```

12. Helper Functions

We now define couple of helper functions which are used to route the flow based on various conditions.

should_retry

- Determines whether SQL execution should be retried based on presence of an error.
- Allows up to **3 retry attempts** using the `iteration` counter.

- Returns "retry", "end", or "success" to control the LangGraph flow.

should_generate_graph

- Checks the `needs_graph` flag set earlier by the visualization decision agent.
- Returns "viz_agent" if a graph should be generated; otherwise returns "skip_graph".
- Controls whether the graph-generating node is executed in the workflow.

check_scope

- Verifies whether the user's question is allowed using `is_in_scope`.
- Returns "in_scope" for valid analytics questions and "out_of_scope" otherwise.
- Determines early termination of the graph when a query is irrelevant or a greeting.

```
#text2sql_agent.py

def should_retry(state: AgentState) -> str:
    """Decide whether to retry after an error"""
    if state.get("error"):
        iteration = state.get("iteration", 0)
        if iteration <= 3:
            return "retry"
        else:
            return "end"
    return "success"

def should_generate_graph(state: AgentState) -> str:
    """Decide whether to generate a graph"""
    if state.get("needs_graph", False):
        return "viz_agent"
    return "skip_graph"

def check_scope(state: AgentState) -> str:
    """Check if question is in scope to continue processing"""
    if state.get("is_in_scope", True):
```

```

return "in_scope"
return "out_of_scope"

```

13. Build the LangGraph Agentic Flow

We now define `create_text2sql_graph` function which:

- **Builds and registers all nodes** in the pipeline, including `guardrails_agent`, `sql_agent`, `execute_sql`, `analysis_agent`, `error_agent`, `decide_graph_need`, and `viz_agent`.
- **Connects nodes using both linear and conditional edges**, powered by helper functions like `check_scope`, `should_retry`, and `should_generate_graph`.
- **Compiles the graph** into an executable state machine that LangGraph can run efficiently and deterministically.

```

#text2sql_agent.py

# Build the LangGraph workflow
def create_text2sql_graph():
    """Create the LangGraph state graph for Text2SQL with graph generation"""

    workflow = StateGraph(AgentState)

    # Add nodes
    workflow.add_node("guardrails_agent", guardrails_agent)
    workflow.add_node("sql_agent", sql_agent)
    workflow.add_node("execute_sql", execute_sql)
    workflow.add_node("analysis_agent", analysis_agent)
    workflow.add_node("error_agent", error_agent)
    workflow.add_node("decide_graph_need", decide_graph_need)
    workflow.add_node("viz_agent", viz_agent)

    # Add edges - start with guardrails check
    workflow.set_entry_point("guardrails_agent")

    # Conditional edge from guardrails - only proceed if in scope
    workflow.add_conditional_edges(
        "guardrails_agent",
        check_scope,
        {
            "in_scope": "sql_agent",
            "out_of_scope": END
        }
    )

```

```

workflow.add_edge("sql_agent", "execute_sql")

# Conditional edge based on execution success
workflow.add_conditional_edges(
    "execute_sql",
    should_retry,
    {
        "success": "analysis_agent",
        "retry": "error_agent",
        "end": "analysis_agent"
    }
)

workflow.add_edge("error_agent", "execute_sql")
workflow.add_edge("analysis_agent", "decide_graph_need")

# Conditional edge for graph generation
workflow.add_conditional_edges(
    "decide_graph_need",
    should_generate_graph,
    {
        "viz_agent": "viz_agent",
        "skip_graph": END
    }
)

workflow.add_edge("viz_agent", END)

return workflow.compile()

# Create the compiled graph
text2sql_graph = create_text2sql_graph()

## Defining the function to generate LangGraph flow visualization

def generate_graph_visualization(output_path: str = "text2sql_workflow.png") ->
    """
    Generate a PNG visualization of the LangGraph workflow.

    Args:
        output_path: Path where the PNG file will be saved (default: "text2sql_

    Returns:
        str: Path to the generated PNG file
    """
    try:
        # Get the graph visualization
        graph_image = text2sql_graph.get_graph().draw_mermaid_png()

        # Save to file
        with open(output_path, "wb") as f:

```

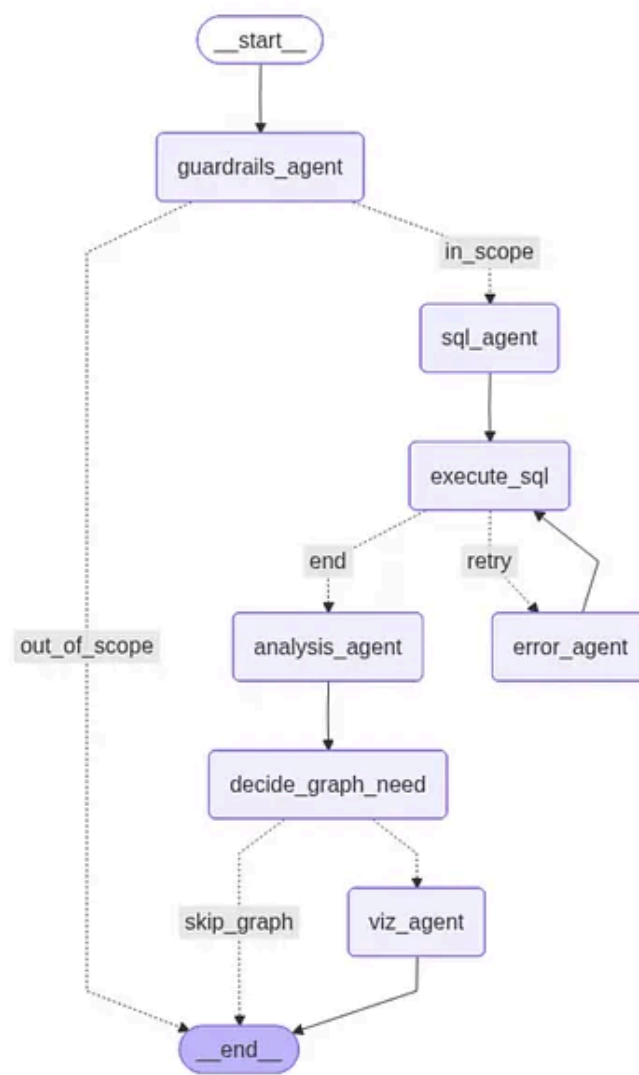
```

        f.write(graph_image)

    print(f"Graph visualization saved to: {output_path}")
    return output_path

except Exception as e:
    print(f"Error generating graph visualization: {e}")
    print("Make sure you have 'pygraphviz' or 'grandalf' installed:")
    print("  pip install pygraphviz")
    print("  or")
    print("  pip install grandalf")
    return None

```



LangGraph Flow

14. Stream the output:

We now define the `process_question_stream` which:

- Streams **node-by-node execution events** (node_start , node_end , error , final) to enable real-time debugging, UI visualization, or timeline playback.
- Initializes a full AgentState , then updates and re-yields state incrementally as each node completes using text2sql_graph.astream_events() .
- Provides a safe async wrapper with structured error handling, ensuring any exceptions are emitted as "error" events rather than crashing the pipeline.

This is the most important step.

We could have used traditional text2sql_graph.invoke() method also which is synchronous and opaque, the process_question_stream() method provides **full transparency and async event-by-event introspection** for debugging and monitoring. You will know the real-time update on the UI for each node.

```
#text2sql_agent.py

async def process_question_stream(question: str):
    """
    Process a natural language question and stream node execution events.
    This is an async generator that yields node events for debugging visualizat

    Yields:
        dict: Event with type ('node_start', 'node_end', 'error', 'final') and
    """
    initial_state = AgentState(
        question=question,
        sql_query="",
        query_result="",
        final_answer="",
        error="",
        iteration=0,
        needs_graph=False,
        graph_type="",
        graph_json="",
        is_in_scope=True
    )

    current_state = initial_state.copy()

    try:
        # Stream events from the graph
        async for event in text2sql_graph.astream_events(
            initial_state,
            config={"recursion_limit": 50},
```

```

        version="v1"
    ):
        event_type = event.get("event")

        # Node start event
        if event_type == "on_chain_start":
            node_name = event.get("name", "")
            if node_name in ["check_guardrails", "generate_sql", "execute_s
                                "handle_error", "decide_graph_need", "generate_g

                yield {
                    "type": "node_start",
                    "node": node_name,
                    "input": current_state
                }

        # Node end event
        elif event_type == "on_chain_end":
            node_name = event.get("name", "")
            if node_name in ["check_guardrails", "generate_sql", "execute_s
                                "handle_error", "decide_graph_need", "generate_g

            output = event.get("data", {}).get("output", {})
            if output:
                current_state.update(output)
                yield {
                    "type": "node_end",
                    "node": node_name,
                    "output": output,
                    "state": current_state.copy()
                }

        # Send final result
        yield {
            "type": "final",
            "result": current_state
        }

    except Exception as e:
        yield {
            "type": "error",
            "error": str(e)
        }

if __name__ == "__main__":
    # Test the agent
    print("=" * 80)
    print("Text2SQL Agent - Use 'chainlit run app.py' to start the web interface")
    print("=" * 80)
    print("\nThis module is meant to be imported and used via the Chainlit app.")
    print("Run: chainlit run app.py")

```

You can skip the next section (Front End) if you are a beginner. Simple write all above functions in a notebook and test your queries there itself.

Notebook is also kept in the [repo here](#).

Testing in Notebook:

Let's define the helping functions to test in the notebook:

```
# define the process question

def process_question(question: str) -> dict:
    """
    Process a natural language question and return the final result.
    This is a simple synchronous function for notebook usage.

    Args:
        question: Natural language question about the e-commerce data

    Returns:
        dict: Final state with answer, SQL query, and graph data if applicable
    """
    initial_state = AgentState(
        question=question,
        sql_query="",
        query_result="",
        final_answer="",
        error="",
        iteration=0,
        needs_graph=False,
        graph_type="",
        graph_json="",
        is_in_scope=True
    )

    try:
        # Invoke the graph
        final_state = text2sql_graph.invoke(
            initial_state,
            config={"recursion_limit": 50}
        )

        return final_state

    except Exception as e:
        return {
            "error": str(e),
            "final_answer": f"An error occurred while processing your question:"
        }
```

```
## Define the test function:

def test_process_question(question: str):

    result = process_question(question)

    print("Final Answer:")
    print(result.get("final_answer", "No answer generated.))

    if result.get("sql_query"):

        print("\nGenerated SQL Query:")
        print(result.get("sql_query", "No SQL query generated.))

    print(f"\nNeeds Graph: {result['needs_graph']}")
    if result['needs_graph']:
        print(f"Graph Type: {result['graph_type']}")

    if result.get("graph_json"):
        import plotly.graph_objects as go
        import plotly.io as pio

        # Load the figure from JSON
        fig = pio.from_json(result['graph_json'])

        # Display in notebook
        fig.show()
    else:
        print("No graph was generated for this question.")
```

Example 1: Hello?

```
question = "Hello!"
test_process_question(question)
```

✓ 1.8s

Final Answer:
Hi! I am your e-commerce assistant. I can answer all the queries related to orders, customers, products, sellers, payments, and reviews between 2016-2028. How can I help you today?

Needs Graph: False
No graph was generated for this question.

Example 2: What are the top 5 states by number of customers?



Example 3: What are the most popular payment methods?



Defining the Front End

We will use `chainlit` library which can give a chatbot front end in few lines of code.

1. Set Page Configuration

We first setup the landing page with welcome messages and sample questions user can ask.

```
# app.py
"""
Chainlit Frontend for Text2SQL Chatbot with Graph Visualization and LangGraph D
"""

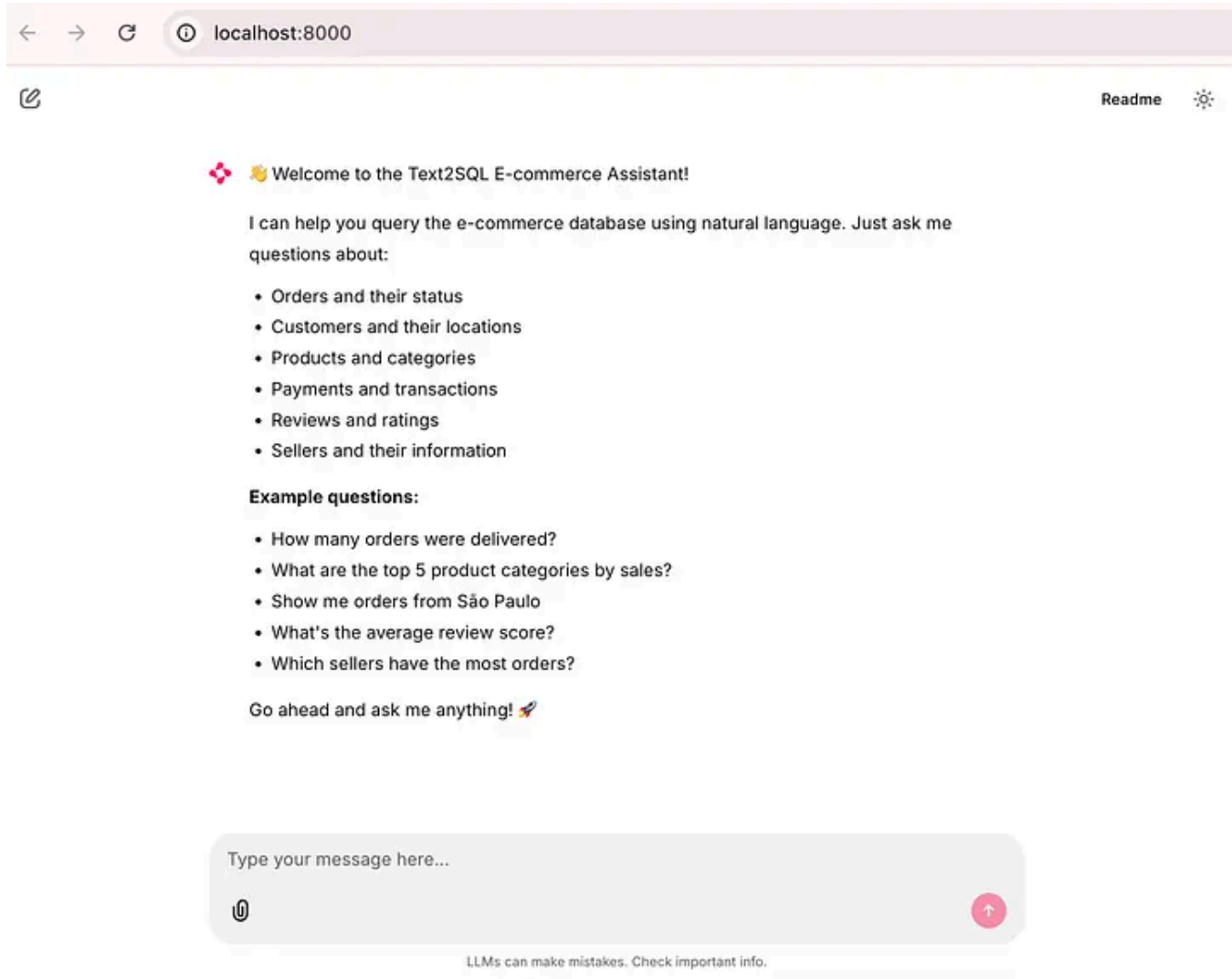
import chainlit as cl
from text2sql_agent import process_question_stream, generate_graph_visualization
import json

##Generate workflow diagram once at module load (optional)
##Uncomment the lines below if you want to generate the diagram:
try:
    workflow_diagram_path = generate_graph_visualization("text2sql_workflow.png")
    if workflow_diagram_path:
        print(f"✅ Workflow diagram generated: {workflow_diagram_path}")
except Exception as e:
    print(f"⚠️ Warning: Could not generate workflow diagram: {e}")

# Set page configuration
@cl.on_chat_start
async def start():
    """Initialize the chat session"""

    await cl.Message(
        content=f"👋 Welcome to the Text2SQL E-commerce Assistant!\n\n"
        "I can help you query the e-commerce database using natural language.\n\n"
        "Just ask me questions about:\n"
        "- Orders and their status\n"
        "- Customers and their locations\n"
        "- Products and categories\n"
        "- Payments and transactions\n"
        "- Reviews and ratings\n"
        "- Sellers and their information\n\n"
        "**Example questions:**\n"
        "- How many orders were delivered?\n"
        "- What are the top 5 product categories by sales?\n"
        "- Show me orders from São Paulo\n"
    ).send()
```

```
)- What's the average review score?\n"- Which sellers have the most orders?\n\n"Go ahead and ask me anything! 🚀"\n).send()
```



Landing Page of the Ecommerce Chatbot

2. Chat Window

This code defines a complete `chainlit` workflow that processes user questions through multiple AI-driven steps such as SQL generation, execution, error handling, and optional graph creation.

It visualizes each step in real time, provides a final answer with optional charts, and ends with a friendly closing message.

This is very powerful as this brings transparency on the table.

```

#app.py
@cl.on_message
async def main(message: cl.Message):
    """Handle incoming messages with debugging visualization"""

    user_question = message.content

    # Create main processing step
    async with cl.Step(name="🤖 Agent Workflow", type="llm") as workflow_step:

        # Dictionary to hold step references
        node_steps = {}
        final_result = None

        try:
            # Stream through the agent execution
            async for event in process_question_stream(user_question):
                event_type = event.get("type")

                # Handle node start
                if event_type == "node_start":
                    node_name = event["node"]
                    node_display_names = {
                        "generate_sql": "📄 Generate SQL Query",
                        "execute_sql": "⚙️ Execute SQL Query",
                        "generate_answer": "💬 Generate Answer",
                        "handle_error": "🔧 Handle Error",
                        "decide_graph_need": "📊 Decide Graph Need",
                        "generate_graph": "✅ Generate Graph"
                    }

                    display_name = node_display_names.get(node_name, node_name)

                    # Create a step for this node
                    node_step = cl.Step(
                        name=display_name,
                        type="tool",
                        parent_id=workflow_step.id
                    )
                    await node_step.send()
                    node_steps[node_name] = node_step

                # Handle node end
                elif event_type == "node_end":
                    node_name = event["node"]
                    output = event["output"]
                    state = event["state"]

                    if node_name in node_steps:
                        node_step = node_steps[node_name]

```

```

# Format output based on node type
output_text = ""

if node_name == "generate_sql":
    sql = output.get("sql_query", "")
    output_text = f"**Generated SQL Query:**\n```\nsql\n{"

elif node_name == "execute_sql":
    if output.get("error"):
        output_text = f"❌ **Error:**\n```\n{output['error']}"
    else:
        result = output.get("query_result", "")
        # Truncate long results for display
        if len(result) > 500:
            result = result[:500] + "\n... (truncated)"
        output_text = f"**Query Results:**\n```\njson\n{"

elif node_name == "generate_answer":
    answer = output.get("final_answer", "")
    output_text = f"**Answer:**\n{answer}"

elif node_name == "handle_error":
    corrected = output.get("sql_query", "")
    iteration = output.get("iteration", 0)
    output_text = f"**Corrected SQL (Attempt {iteration})**\n\n{corrected}"

elif node_name == "decide_graph_need":
    needs_graph = output.get("needs_graph", False)
    graph_type = output.get("graph_type", "")
    if needs_graph:
        output_text = f"✅ **Graph Needed:** {graph_type}"
    else:
        output_text = f"❌ **No graph needed** for this query"

elif node_name == "generate_graph":
    has_graph = bool(output.get("graph_json"))
    if has_graph:
        output_text = f"✅ Graph generated successfully"
    else:
        output_text = f"⚠️ Graph generation skipped"

# Update the step with output
node_step.output = output_text
await node_step.update()

# Handle final result
elif event_type == "final":
    final_result = event["result"]

# Handle errors
elif event_type == "error":
    error_msg = event["error"]
    workflow_step.output = f"❌ **Error:** {error_msg}"

```

```

        await workflow_step.update()
        return

    # Mark workflow as complete
    workflow_step.output = "✅ Workflow completed successfully"
    await workflow_step.update()

except Exception as e:
    workflow_step.output = f"❌ **Unexpected Error:** {str(e)}"
    await workflow_step.update()
    raise

# Now send the final response outside the workflow step
if final_result:
    # Build the response content
    # Only show SQL query if it exists and is not empty
    if final_result.get('sql_query') and final_result['sql_query'].strip():
        response_content = f"""**Generated SQL Query:**

```sql
{final_result['sql_query']}
```

**Answer:**
{final_result['final_answer']}
"""
    else:
        # For greetings or out of scope messages, just show the answer
        response_content = final_result['final_answer']

    # If there was an error, include it
    if final_result.get('error'):
        response_content += f"\n\n⚠️ **Note:** {final_result['error']}"

    # Send text response
    await cl.Message(content=response_content).send()

    # Send graph if available - use Chainlit's native Plotly element
    if final_result.get('needs_graph') and final_result.get('graph_json'):
        # Send interactive Plotly visualization using Chainlit's Plotly element
        import plotly.graph_objects as go

        # Parse the JSON back to a figure
        fig = go.Figure(json.loads(final_result['graph_json']))

        graph_element = cl.Plotly(
            name=f"{final_result.get('graph_type', 'chart')}_visualization"
            figure=fig,
            display="inline"
        )

        await cl.Message(
            content=f"📊 **Interactive Visualization** ({final_result.get('graph_type', 'chart')})"
            elements=[graph_element]

```

```
) .send()
```

```
@cl.on_chat_end
async def end():
    """Handle chat end"""
    await cl.Message(content="Thanks for using the Text2SQL Assistant! 🙌").send()

if __name__ == "__main__":
    # Run with: chainlit run app.py
    pass
```

localhost:8000

How many orders were delivered?

Used 🧠 Agent Workflow ^

Used guardrails_agent

Used sql_agent

Used ⚙️ Execute SQL Query ▾

Used analysis_agent

Used 📊 Decide Graph Need ▾

✅ Workflow completed successfully

Generated SQL Query:

sql

```
SELECT COUNT(*) AS total_delivered_orders
FROM orders
WHERE order_status = 'delivered';
```

Answer:

Based on the query results, here is the answer to your question:

- A total of **96,478 orders** were delivered.

Type your message here...

LLMs can make mistakes. Check important info.

Chatbot Question and Answer with Intermediate Steps

Run the App

You can clone the repo from your terminal using the following command:

```
git clone https://github.com/alphaiterations/multi-agent-chatbot.git
```

Then navigate into the folder:

```
cd multi-agent-chatbot
```

Now create a virtual environment `.venv`

```
python3 -m venv .venv
```

Activate the virtual environment

```
source .venv/bin/activate
```

Install the required libraries

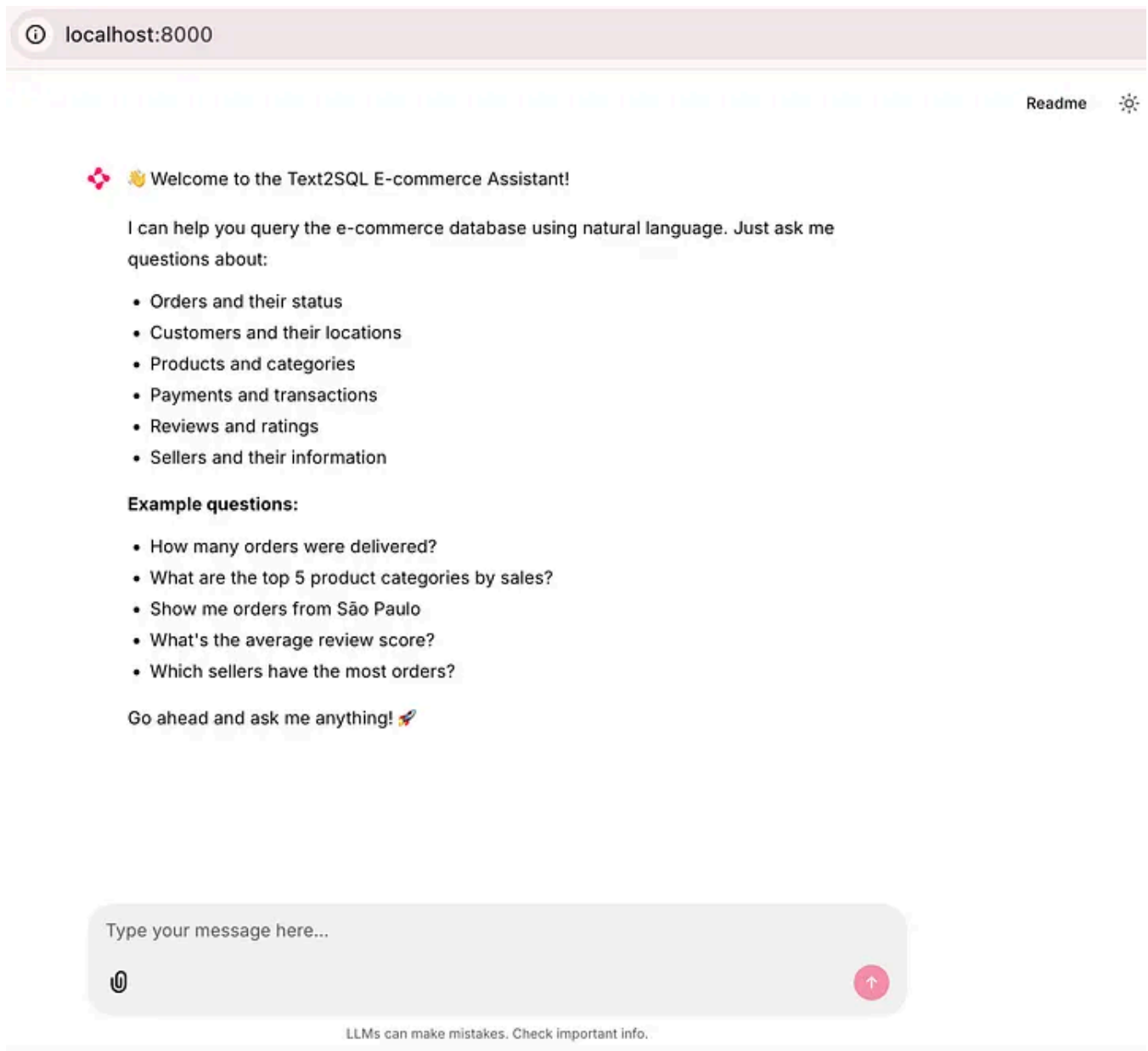
```
pip install -r requirements.txt
```

Run the app:

```
chainlit run app.py
```

```
#output
2025-11-19 09:14:29 - Loaded .env file
Graph visualization saved to: text2sql_workflow.png
✅ Workflow diagram generated: text2sql_workflow.png
2025-11-19 09:14:31 - Your app is available at http://localhost:8000
```

This will open the chatbot at this link <http://localhost:8000>



Welcome Screen on the Chatbot

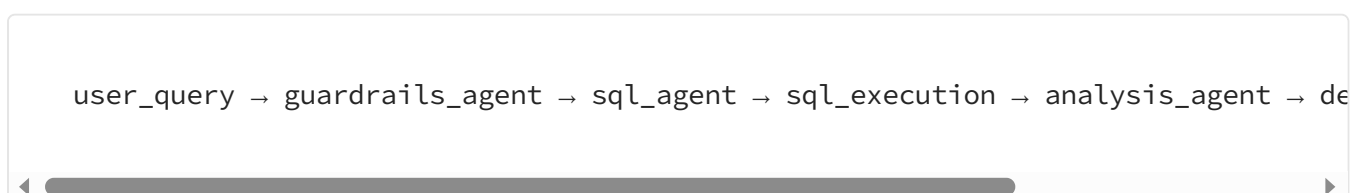
That's it. We are now ready to start interacting with the chatbot.

Testing the Chatbot

1. Simple Queries

Query 1: *How many orders were delivered in 2017?*

Flow: Since it is single number answer hence the graph is not needed here.



localhost:8000

How many orders were delivered in 2017?

Used 🧠 Agent Workflow ^

Used guardrails_agent

Used sql_agent

Used ⚙️ Execute SQL Query ▾

Used analysis_agent

Used 📊 Decide Graph Need ▾

✅ Workflow completed successfully

Generated SQL Query:

sql

```
SELECT COUNT(*) AS total_delivered_orders
FROM orders
WHERE order_status = 'delivered'
AND order_purchase_timestamp BETWEEN '2017-01-01' AND '2017-12-31';
```

Answer:

In 2017, the total number of orders that were delivered is as follows:

- **Total Delivered Orders:** 43,357

This indicates a significant volume of successful deliveries throughout the year.

Type your message here...

LLMs can make mistakes. Check important info.

Query 2: Can you tell me the yearly breakdown of total revenue generated?

Since we are asking for the yearly breakdown, the line chart makes complements the answer beautifully.

user_query → guardrails_agent → sql_agent → sql_execution → analysis_agent → de

Can you tell me the yearly breakdown of total revenue generated?

Used 🧠 Agent Workflow ^

Used guardrails_agent

Used sql_agent

Used ⚙️ Execute SQL Query ^

Used analysis_agent

Used 📊 Decide Graph Need ^

Used viz_agent

✅ Workflow completed successfully

Generated SQL Query:

```
sql
SELECT strftime('%Y', order_purchase_timestamp) AS year, SUM(payment_value) AS total_revenue
FROM orders
JOIN order_payments ON orders.order_id = order_payments.order_id
GROUP BY year
ORDER BY year
LIMIT 10;
```

Answer:

Here's the yearly breakdown of total revenue generated based on the query results:

- 2016: \$59,362.34
- 2017: \$7,249,746.73
- 2018: \$8,699,763.05

From the data, we can see a significant increase in total revenue from 2016 to 2017, and again from 2017 to 2018.

Interactive Visualization (Line Chart)

Hover over the chart for details, zoom, and pan!



2. Complex Queries

Query 1: (Asking multiple questions)

How much revenue was generated in 2018? And How many orders were placed in 2017?

This generates two SQL queries and system executes both the queries and gives the answers properly.

user_query → guardrails_agent → sql_agent → sql_execution_1 → sql_execution_2



localhost:8000

How much revenue was generated in 2018? And How many orders were placed in 2017?



Used 🧠 Agent Workflow ^

Used guardrails_agent

Used sql_agent

Used ⚙️ Execute SQL Query v

Used analysis_agent

Used 📊 Decide Graph Need v

✅ Workflow completed successfully



Generated SQL Query:

sql



```
SELECT SUM(payment_value) AS total_revenue
FROM order_payments
JOIN orders ON order_payments.order_id = orders.order_id
WHERE order_approved_at BETWEEN '2018-01-01' AND '2018-12-31';

SELECT COUNT(order_id) AS total_orders
FROM orders
WHERE order_purchase_timestamp BETWEEN '2017-01-01' AND '2017-12-31';
```

Answer:

Based on the query results, here are the insights regarding revenue and orders:

1. Revenue Generated in 2018:

The total revenue generated in 2018 was **\$8,685,499.54**.

2. Orders Placed in 2017:

A total of **45,027 orders** were placed in 2017.

If you have any further questions or need additional insights, feel free to ask!



Type your message here...



LLMs can make mistakes. Check important info.

Query 2: *Find the top 10 customers who spent the most money across all their orders.*

This uses multiple tables to generate the query. Below is the flow:

user_query → guardrails_agent → sql_agent → sql_execution → analysis_agent → de

Find the top 10 customers who spent the most money across all their orders.

Used 🧠 Agent Workflow ^

Used guardrails_agent

Used sql_agent

Used ⚙️ Execute SQL Query ▾

Used analysis_agent

Used 📊 Decide Graph Need ▾

Used viz_agent

✅ Workflow completed successfully

🔥 Generated SQL Query:

sql

```
SELECT c.customer_id, SUM(p.payment_value) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_payments p ON o.order_id = p.order_id
GROUP BY c.customer_id
ORDER BY total_spent DESC
LIMIT 10;
```

Answer:

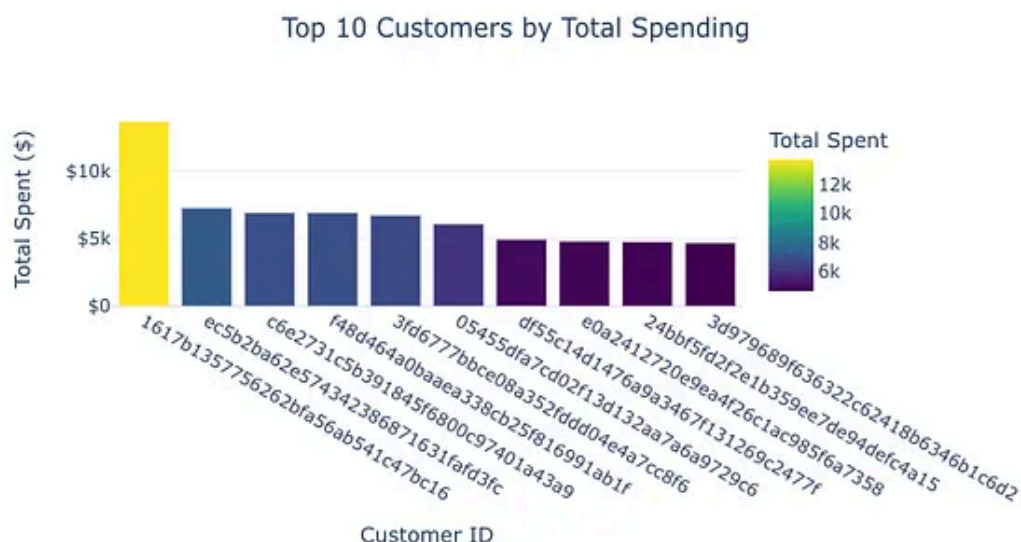
Here are the top 10 customers who spent the most money across all their orders:

1. **Customer ID:** 1617b1357756262bfa56ab541c47bc16
 - **Total Spent:** \$13,664.08
2. **Customer ID:** ec5b2ba62e574342386871631fafd3fc
 - **Total Spent:** \$7,274.88
3. **Customer ID:** c6e2731c5b391845f6800c97401a43a9
 - **Total Spent:** \$6,929.31
4. **Customer ID:** f48d464a0baaea338cb25f816991ab1f
 - **Total Spent:** \$6,922.21
5. **Customer ID:** 3fd6777bbce08a352fddd04e4a7cc8f6
 - **Total Spent:** \$6,726.66
6. **Customer ID:** 05455dfa7cd02f13d132aa7a6a9729c6
 - **Total Spent:** \$6,081.54
7. **Customer ID:** df55c14d1476a9a3467f131269c2477f
 - **Total Spent:** \$4,950.34
8. **Customer ID:** e0a2412720e9ea4f26c1ac985f6a7358
 - **Total Spent:** \$4,809.44
9. **Customer ID:** 24bbf5fd2f2e1b359ee7de94defc4a15
 - **Total Spent:** \$4,764.34
10. **Customer ID:** 3d979689f636322c62418b6346b1c6d2
 - **Total Spent:** \$4,681.78

These customers represent the highest spending in your database, with the top spender exceeding \$13,000 in total purchases.

**Interactive Visualization (Bar Chart)**

Hover over the chart for details, zoom, and pan!



Finally we have completed our multi-agent e-commerce chatbot project using LangGraph, OpenAI and ChainLit leveraging the power of Agentic AI. It wasn't that difficult right?

Refer to the complete [github repo here](#).

Feel free to share your comments if you have any doubts / feedback.

Advanced Features:

1. **Intelligent Chat History Management:** Persistent conversation storage with contextual follow-up capabilities
2. **Personalized User Profiling:** Adaptive learning of user preferences and analytical patterns for customized responses
3. **Advanced Session Management:** Multi-session state persistence with collaborative sharing and recovery features
4. **Smart Query Optimization:** AI-powered query caching, performance analytics, and adaptive execution planning
5. **Enterprise Security Framework:** Role-based access control, comprehensive audit logging, and compliance-ready data governance
6. **Enhanced NLP Capabilities:** Multi-language support, domain-specific vocabulary learning, and intelligent clarification dialogues

Conclusion

This Text2SQL Agent system represents a significant advancement in natural language data interaction. By combining LangGraph's stateful workflows with specialized AI agents, it creates a robust, intelligent system that goes beyond simple query conversion to provide comprehensive data analysis and visualization.

The project successfully demonstrates Agentic AI & Multi Agent architecture can solve complex data interaction problems while maintaining production-grade reliability and excellent user experience. It serves as a strong foundation for building more advanced conversational data analytics platforms.

References:

1. [Brazilian E-Commerce Public Dataset by Olist](#) [Kaggle]

2. [Chainlit Documentation](#)

3. [Multi-Agent E-Commerce Chatbot with LangGraph and OpenAI Github Repo](#)

Thank you for reading the article.

Growth isn't comfortable, it's intentional. [Follow me](#) if you can handle more.

You can also follow me on [Linkedin](#) to get regular updates or further collaboration.

Learning doesn't end here. Please refer to my other articles:

1. [Build Agentic RAG using LangGraph](#)
2. [Practical Guide to Using ChromaDB for RAG and Semantic Search](#)
3. [Reading Images with GPT-4o: The Future of Visual Understanding with AI](#)
4. [Agentic AI Project: Build Mini Perplexity AI Chatbot : Step by Step Guide \[Code Included\]](#)
5. [Agentic AI: Build ReAct Agent using LangGraph](#)
6. [Agentic AI Project: Build a multi-agent system with LangGraph and OpenAI API](#)
7. [Building an AI Agent with Model Context Protocol \(MCP\): A Complete Guide](#)
8. [TOON vs JSON: A Comprehensive Performance Comparison](#)
9. [Building an Intelligent Resume Transformation Agent Powered by LangGraph and gpt-4o-mini](#)
10. [Agentic AI Project: Build a Customer Service Chatbot for a Clinic](#)
11. [Agentic AI: Vectorless RAG: How I Built a RAG System Without Embeddings, Databases, or Vector Similarity](#)
12. [Agentic AI Project: MLflow Observability for Generative AI — A Deep Dive with Text2SQL + RAG + WebSearch using LangGraph, Open AI](#)

[Agentic Ai](#)[Data Science](#)[Generative Ai Use Cases](#)[Langgraph](#)[Chatbots](#)



Following

Published in Towards AI

138K followers · Last published 2 hours ago

We build Enterprise AI. We teach what we learn. Join 100K+ AI practitioners on Towards AI Academy. Free: 6-day Agentic AI Engineering Email Guide: <https://email-course.towardsai.net/>



Follow



Written by Alpha Iterations

1.2K followers · 29 following

Generative AI | Agentic AI | AI Applications

Responses (8)



Datachamp

What are your thoughts?



Cleber Veiga

Nov 28, 2025



Great article! I implemented a similar Agentic Text2SQL project recently, but I faced a challenge with a legacy database containing thousands of messy tables.

A highly effective strategy we found was encapsulating the database complexity into SQL... [more](#)



10

[Reply](#)**Dhilip Kumar**

Nov 27, 2025



Very good Narrative.. and easy to follow..

Have you tested complex joins like top 10 orders who's order value greater x amount in a given customer country...



2

[Reply](#)**Minalu Mesele**

Apr 2



Very Good idea, yes the sales, marketing staff and even the owner of businesses need to check any data in any format he/she wants to see. Having such natural language interaction of users to database would add value to any data intensive systems! Thanks for sharing the idea.

[Reply](#)[See all responses](#)

